

- **ReactJS Concept Guide: 25+ Questions & Answers**

1. What is ReactJS?

Answer:

ReactJS is a **JavaScript library for building user interfaces**. It is maintained by Facebook and focuses on creating reusable UI components for **single-page applications (SPAs)**. React uses a **virtual DOM** for efficient rendering.

2. What is the Virtual DOM?

Answer:

The **Virtual DOM** is a lightweight copy of the actual DOM. React updates the Virtual DOM first and then **diffs** it with the real DOM to apply **minimal changes**, improving performance.

3. What are React components?

Answer:

Components are **reusable building blocks** of a React app.

- **Functional Components:** Use functions and hooks.
 - **Class Components:** Use ES6 classes and lifecycle methods.
-

4. What is JSX?

Answer:

JSX (JavaScript XML) is a **syntax extension** that allows you to write **HTML-like code inside JavaScript**. React converts JSX into `React.createElement()` calls.

5. What are props in React?

Answer:

Props (properties) are **read-only inputs** passed from parent to child components. They help make components **reusable**.

6. What is state in React?

Answer:

State is **mutable data managed within a component**. Changing the state triggers a **re-render** of the component.

7. Difference between State and Props

Answer:

Feature	State	Props
Mutable	Yes	No
Managed by	Component itself	Parent component
Purpose	Dynamic data	Pass data

8. What are React Hooks?

Answer:

Hooks are **functions that let functional components use state and lifecycle features**. Examples:

- `useState()`
 - `useEffect()`
 - `useContext()`
-

9. Explain `useState` Hook

Answer:

`useState()` lets you **add state to functional components**.

```
const [count, setCount] = useState(0);
```

- `count` = current value
 - `setCount` = function to update value
-

10. Explain useEffect Hook

Answer:

useEffect() manages **side effects** like API calls, subscriptions, or DOM updates. It replaces most **lifecycle methods**.

```
useEffect(() => {  
  
  console.log('Component mounted or updated');  
  
}, [dependencies]);
```

11. What are React lifecycle methods?

Answer:

Lifecycle methods exist in **class components**:

- componentDidMount() – Runs after component mounts
 - componentDidUpdate() – Runs after update
 - componentWillUnmount() – Runs before unmount
-

12. What is a controlled component?

Answer:

A controlled component has **form input values controlled by React state**.

```
<input value={stateValue} onChange={handleChange} />
```

13. What is an uncontrolled component?

Answer:

Uncontrolled components manage their **own internal state**, using **refs** to access values.

14. What is React Router?

Answer:

React Router is a library for **client-side routing**, allowing **SPA navigation** without page reloads.

- <BrowserRouter>
 - <Route>
 - <Link>
-

15. What is the difference between React and Angular?

Answer:

Feature	React	Angular
Type	Library	Framework
Data Binding	One-way	Two-way
DOM	Virtual DOM	Real DOM
Learning Curve	Moderate	Steep

16. What is Redux?

Answer:

Redux is a **state management library** for React apps. It centralizes state in a **store**, making it predictable.

17. What are React keys?

Answer:

Keys help React **identify which items in a list have changed** for efficient re-rendering. They must be **unique and stable**.

18. What is context in React?

Answer:

Context allows **passing data through the component tree without props drilling**.

```
const ThemeContext = React.createContext();
```

19. What are higher-order components (HOC)?

Answer:

HOCs are **functions that take a component and return a new enhanced component.**

```
const EnhancedComponent = withHOC(MyComponent);
```

20. What is React Fragment?

Answer:

Fragments let you **group multiple elements without adding extra DOM nodes:**

```
<React.Fragment>
```

```
  <Child1 />
```

```
  <Child2 />
```

```
</React.Fragment>
```

21. How to handle events in React?

Answer:

React uses **camelCase events** and **functions instead of strings:**

```
<button onClick={handleClick}>Click Me</button>
```

22. Difference between class and functional components

Answer:

Feature	Class	Functional
Syntax	ES6 class	Function
State	Yes	Yes (with hooks)

Feature	Class	Functional
Lifecycle	Yes	Use useEffect

23. What is lazy loading in React?

Answer:

Lazy loading loads **components only when needed**, improving performance:

```
const LazyComponent = React.lazy(() => import('./Component'));
```

24. What is React Suspense?

Answer:

Suspense is used with **lazy loading** to show a fallback UI until the component **finishes loading**:

```
<Suspense fallback={<div>Loading...</div>}>
```

```
  <LazyComponent />
```

```
</Suspense>
```

25. How to optimize React performance?

Answer:

- Use **React.memo** to memoize components
 - Use **useCallback** and **useMemo** hooks
 - Split code using **lazy loading**
 - Avoid unnecessary state updates
-

26. Difference between virtual DOM and real DOM

Answer:

- **Virtual DOM:** Lightweight copy, faster updates, React uses diffing
- **Real DOM:** Browser DOM, slower for updates